NEURIPS 2026 · LP4FM WORKSHOP · POSTER

ReconSpan: Reconstruction-Guided Adaptive Latent Tokenization

Lixing Li · Cornell University

TL;DR ReconSpan cuts text into chunks that a backward decoder can reconstruct from a single contextual prefix code, and keeps one such code as the latent token for each chunk. At matched per-document latent-token counts, these boundaries preserve more source tokens in native reconstruction than equal-spacing, surprisal, or random boundaries.

6.50 – 12.28
mean base tokens per latent token from one trained autoencoder, set by the stopping rule

.907
of tokens lie in perfectly reconstructed chunks at matched chunk count (equal spacing .816)

74.3 vs 10.0
Llama-3-8B reader on latents: AG News topic accuracy vs LAMBADA exact word (%)

MOTIVATION

Where should a tokenizer spend its positions?

1 PROBLEM

- Subword tokenizers fix their units **before** the model runs, largely from corpus-level frequency statistics, so granularity is not conditioned on the actual input.
- Byte- and character-level inputs avoid a fixed vocabulary but produce substantially longer sequences.
- **Adaptive latent tokenization** groups neighboring units into input-dependent spans and represents each span with one continuous latent token.
- The chunking rule is therefore central: boundaries decide where new latent positions, and so representation and computation, are allocated.

Idea. Place each boundary where a decoder can no longer reconstruct the span from a single code: difficult spans get shorter chunks, easier spans get longer ones.

2 ALLOCATION RULES IN RELATED WORK

BOUNDARY SIGNAL	EXAMPLES
Fixed position	MEGABYTE; Extensible Tokenization
Forward predictive entropy	Byte Latent Transformer; Dynamic Token Pooling
Semantic similarity	SemToken
Learned boundary scores	Dynamic Token Pooling; H-Net; Charformer; FLEXITOKENS
Backward reconstruction fidelity	**ReconSpan (this work)**

The contribution is the allocation criterion and a characterization of the resulting latent tokens, not continuous tokens or adaptive segmentation by themselves. ReconSpan runs over an existing subword sequence: it is a second-stage latent tokenizer, not a byte-to-text replacement.

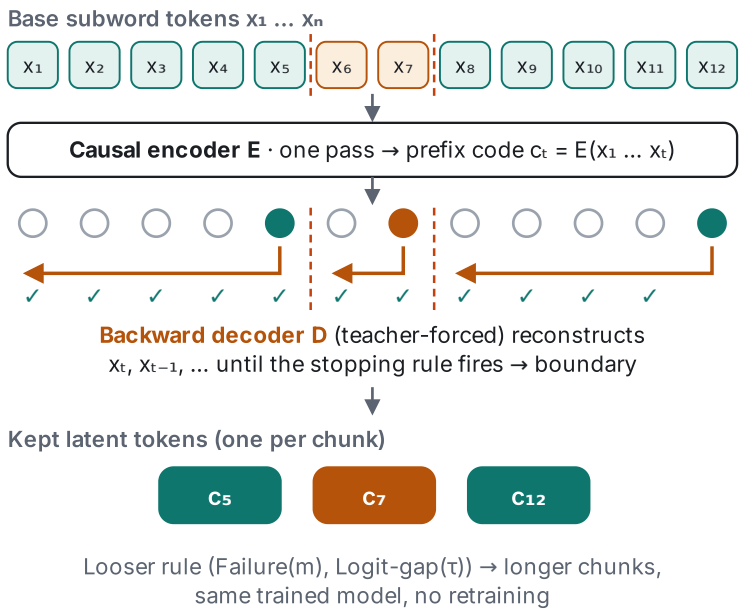
METHOD

Chunk where one code stops reconstructing

1 CHUNKING ALGORITHM

1. One causal encoder pass gives a prefix code $c_t = E(x_{1:t})$ at every position.
2. From the last position, teacher-force the backward decoder D from c_t against x_t, x_{t-1}, … until the stopping rule fires.
3. Accept the reconstructed suffix as a chunk and keep c_t as its latent token; resume at the first unreconstructed position (a one-token chunk if it fails at once).
4. Return the kept boundary codes in chronological order.

2 PIPELINE SCHEMATIC



Illustrative chunk sizes, not a paper figure. Two stopping families: **Failure(*m*)** stops at the *m*th wrong token; **Logit-gap(*τ*)** stops when the summed gap between the top logit and the true token's logit exceeds τ.

3 MODEL & TRAINING

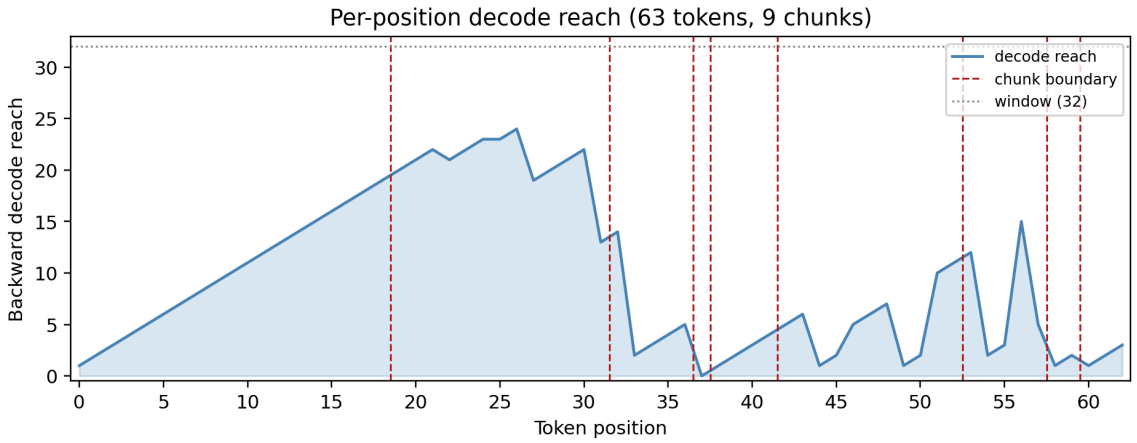
- **Encoder:** Pythia-410M Transformer; its final 1024-dimensional hidden state is the code.
- **Decoder:** Mamba2-130M backward model; the code is up-projected to its per-layer initial SSM states.
- **Objective:** reconstruction only, decoding a window (usually a suffix) in reverse; no length target.
- **Data:** FineWeb; 7B-token decoder pretrain with frozen encoder, then 3B tokens of joint training.

Why backward? Decoding newest-first, the first failure directly measures how far back one code reaches.

METHOD IN ACTION

Reconstruction reach sets the boundaries

1 ONE 63-TOKEN PASSAGE UNDER FAILURE(1)



Water boils at 100 degrees Celsius, and Genghis Khan founded the Mongol | Empire in 1206. The clarinet has a single-reed | mouthpiece, while photos | ynthesis | converts sunlight into glucose | . Napoleon was exiled to Elba in 1814 | , but the liver performs | over 500 | metabolic functions.

Paper appendix figure: decode reach from every prefix code; red lines mark the 8 interior boundaries. Within-word splits show the example does not imply linguistic alignment.

2 HOW FAR CAN ONE CODE REACH?

Mean longest exactly reconstructed suffix (tokens) from one 1024-dim code of an ℓ-token window, 500 held-out Wikipedia windows.

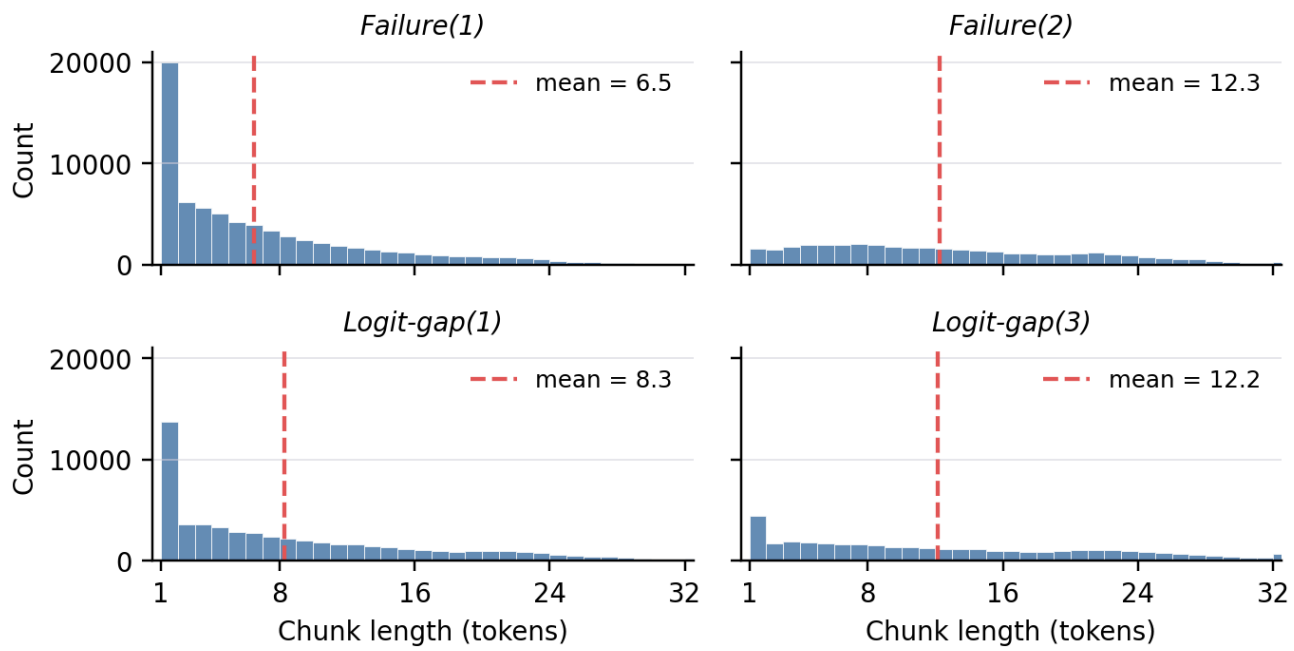
ℓ	RECONSPAN	SONAR
8	6.47	5.00
16	10.86	7.94
32	9.94	7.24
64	8.15	1.16
128	8.26	0.05
256	7.42	0.03

Short-range reach stays stable as the window grows, while SONAR's suffix collapses beyond $\ell = 32$. That stable reach is what chunking relies on.

RESULT 1 · ADAPTIVE SPAN ALLOCATION

One trained model, a dial for granularity

1 CHUNK-LENGTH DISTRIBUTIONS



Four stopping rules applied post hoc to the same model on the same 500 Wikipedia documents; dashed lines mark the mean. The small pile-up at 32 comes from the $W = 32$ decode window (<1% of chunks).

2 MEAN CHUNK LENGTH

STOPPING RULE	BASE TOKENS
Failure(1)	6.50
Logit-gap(1)	8.31
Logit-gap(3)	12.17
Failure(2)	12.28

9.56

mean *Failure(1)* chunk length over 2 million FineWeb documents: realized averages vary with the input

28.1% of chunks

are single tokens under *Failure(1)*, carrying only 4.3% of the text (4.1% of chunks under *Failure(2)*)

Same budget, better boundaries

1 NATIVE ROUND TRIP ON 500 WIKIPEDIA DOCUMENTS

POLICY / RULE	MEAN LEN.	EXACT CHUNK	EXACT TOKEN	SUFFIX	PPL ↓
Same per-document chunk count as Failure(1)					
ReconSpan, *Failure(1)*	**6.50**	**.797**	**.907**	**.944**	**15.95**
equal spacing	6.50	.809	.816	.895	15.75
surprisal	6.50	.802	.617	.773	19.50
random (matched)	6.50	.794	.618	.767	20.03
Looser ReconSpan rules (same model)					
Logit-gap(1)	8.31	.730	.809	.890	16.84
Logit-gap(3)	12.17	.553	.541	.715	20.37
Failure(2)	12.28	.526	.536	.740	18.43

Each chunk is re-encoded in isolation and decoded. Exact token: share of tokens in perfectly reconstructed chunks. Suffix: share in each chunk's correct newest-first run. PPL: Qwen2.5-1.5B, separate WikiText run.

2 READING THE TABLE

.907 vs **.816**

exact-token agreement, ReconSpan vs equal spacing (surprisal .617, random .618)

Scope. Equal spacing is slightly higher on unweighted exact chunk (.809 vs .797) and slightly lower in PPL (15.75 vs 15.95). The win is on **token-weighted** native reconstruction, not uniform dominance.

Where boundaries fall, not only how many there are, decides how much text survives. Across rule families, mean chunk length largely predicts quality.

RESULT 3 · READING THE LATENT TOKENS

Readers recover topic, not exact detail

1 DOWNSTREAM READOUT FROM LATENT TOKENS (%)

TASK	READER	READOUT	CONTROL	TEXT	ROUNDTRIP
AG News topic, 4-way	Pythia-410M	52.5*	26.4	53.0	52.8
	Llama-3-8B	**74.3***	**24.5**	**72.6**	**73.0**
	Llama-3-8B + task FT	82.7*	24.2	72.6	73.0
LAMBADA exact last word	Pythia-410M	5.1*	1.8	51.0	50.2
	Llama-3-8B	**10.0***	**5.5**	**75.8**	**72.5**
	Llama-3-8B + task FT	20.1*	6.2	75.8	72.5
HotpotQA answer EM	Pythia-410M	0.0	1.0	7.0	6.0
	Llama-3-8B	17.0	18.0	36.0	33.0
	Llama-3-8B + task FT	30.0	23.0	36.0	33.0

Failure(1) codes. Control: another example's codes. Text: raw context to the base model. Roundtrip: autoencoder-reconstructed text. * above control at 95% confidence.

2 WHAT THE READERS SHOW

- The Llama reader reaches raw-text-level topic accuracy on AG News, but LAMBADA stays far below the round-trip reference.
- Generic readout on HotpotQA does not beat the shuffled control. Task fine-tuning helps all three tasks.
- Readers trained only on Failure(1) codes hold up at Failure(2): AG News 74.6 at mean length 12.9.

CONTINUATION SOURCE	PPL ↓
Human continuation	11.95
Llama-3-8B reader	**11.78**
Pythia-410M reader	13.94

Fluent, but fluency is not faithfulness: samples keep topic and register while exact content is hard to access.

DISCUSSION

Takeaways, limitations, and next steps

1 TAKEAWAYS

- Backward reconstruction fidelity is a viable allocation criterion for adaptive latent tokenization.
- The stopping rule is a post-training granularity dial on one model, with no retraining.
- At matched per-document budgets, ReconSpan boundaries lead on token-weighted native reconstruction.
- The native route retains information that current readers do not fully access.

2 LIMITATIONS

- **Reader access:** direct-code readers stay weak on exact content and retrieval.
- **Short chunks:** 28.1% of *Failure(1)* chunks cover one token.
- **Backward decoder:** none is readily available, so it is trained from scratch.
- **Selection cost:** about 12 hours to build the 2M-document reader corpus; fewer latent positions do not by themselves mean lower end-to-end compute.
- **Scope:** runs over subword tokens; no direct comparison with BLT or H-Net.

3 NEXT STEPS

- Stronger readers or objectives aimed at exact retrieval.
- A semantic-clustering objective to organize code geometry.
- Test whether boundaries align with syntax, discourse, or information density.
- Evaluate as context compression: prefill latency, KV-cache memory, amortized selection cost.